Pintos Project 2: User Programs — Project Handout

1. Overview

Project 2 moves Pintos from “kernel-only demos” to running **user programs**. The baseline kernel can load and start user processes. In this lab, you will implement the user-facing OS interface: process control, and safe handling of user memory. Primary work happens in `src/userprog/`, with supporting changes across other directories.

2. Learning Goals

By the end of this project, you should be able to:

- Explain the boundary between **user mode** and **kernel mode** in Pintos.
- Set up the initial **user stack** correctly to support command-line argument passing.
- Implement two simple **system calls**.
- Safely access **user memory** from the kernel (avoid kernel crashes caused by invalid user pointers).
- Support basic **process management** (halt/exit).

3. Where You Will Work (Key Source Areas)

You will primarily read/modify code in:

- `src/userprog/`
  - `process.c/.h` — process creation and ELF loading
  - `syscall.c/.h` — system call entry and dispatch
  - `src/threads/` — synchronization primitives and helper utilities

4. Background

4.1 Source Files

The easiest way to get an overview of the programming you will be doing is to simply go over each part you'll be working with. In `userprog/`, you'll find a small number of files, but here is where the bulk of your work will be:

process.c

process.h

Loads ELF binaries and starts processes.

pagedir.c

pagedir.h

A simple manager for 80x86 hardware page tables.
Although you probably won't want to modify this code for this project, you may want to call some of its functions.

syscall.c
syscall.h

Whenever a user process wants to access some kernel functionality, it invokes a system call. This is a skeleton system call handler.
Currently, it just prints a message and terminates the user process. In part 2 of this project you will add code to do everything else needed by system calls.

exception.c
exception.h

When a user process performs a privileged or prohibited operation, it traps into the kernel as an "exception" or "fault." These files handle exceptions.
Currently all exceptions simply print a message and terminate the process.
Some, but not all, solutions to project 2 require modifying `page_fault()` in this file.

gdt.c
gdt.h

The 80x86 is a segmented architecture. The Global Descriptor Table (GDT) is a table that describes the segments in use. These files set up the GDT.
You should not need to modify these files for any of the projects. You can read the code if you're interested in how the GDT works.

tss.c
tss.h

The Task-State Segment (TSS) is used for 80x86 architectural task switching.
Pintos uses the TSS only for switching stacks when a user process enters an interrupt handler, as does Linux.
You should not need to modify these files for any of the projects. You can read the code if you're interested in how the TSS works.

4.2 Using the File System

You will need to interface to the file system code for this project, because user programs are loaded from the file system.
However, the focus of this project is not the file system, so we have provided a simple but complete file system in the filesys/ directory.
You will want to look over the filesys.h and file.h interfaces to understand how to use the file system, and especially its many limitations.

There is no need to modify the file system code for this project, and so we recommend that you do not. Working on the file system is likely to distract you from this project's focus.

4.2.1 Limitations

Although we will not be improving the file system implementation in a later project, proper use of the file system routines now will still make your code more robust and easier to extend. Until then, you will have to tolerate the following limitations:

- **No internal synchronization. Concurrent accesses will interfere with one another. You should use synchronization to ensure that only one process at a time is executing file system code.**
- File size is fixed at creation time. The root directory is represented as a file, so the number of files that may be created is also limited.
- File data is allocated as a single extent, that is, data in a single file must occupy a contiguous range of sectors on disk. External fragmentation can therefore become a serious problem as a file system is used over time.
- No subdirectories.
- File names are limited to 14 characters.
- A system crash mid-operation may corrupt the disk in a way that cannot be repaired automatically. There is no file system repair tool anyway.

4.2.2 Features

One important feature is included:
Unix-like semantics for *filesys_remove()* are implemented. That is, if a file is open when it is removed, its blocks are not deallocated and it may still be accessed by any threads that have it open, until the last one closes it.

4.2.3 Create your simulated disk/file system

You need to be able to create a simulated disk with a file system partition.
The `pintos-mkdisk` program provides this functionality.
From the `userprog/build` directory, execute `pintos-mkdisk filesys.dsk --filesys-size=2`. This command creates a simulated disk named `filesys.dsk` that contains a 2 MB Pintos file system partition.
Then format the file system partition by passing `-f -q` on the kernel's command line: `pintos -- -f -q.` The `-f` option causes the file system to be formatted, and `-q` causes Pintos to exit as soon as the format is done.
You'll need a way to copy files in and out of the simulated file system.
The `pintos -p` ("put") and `-g` ("get") options do this.

- To copy files into the Pintos file system, use the command `pintos -p file -- -q`. (The `--` is needed because `-p` is for the pintos script, not for the simulated kernel.)
- To copy it to the Pintos file system under the name newname, add -a newname: `pintos -p file -a newname -- -q`.
- The commands for copying files out of a VM are similar, but substitute `-g` for `-p`.

- Incidentally, these commands work by passing special commands extract and append on the kernel's command line and copying to and from a special simulated "scratch" partition. If you're very curious, you can look at the pintos script as well as `filesys/fsutil.c` to learn the implementation details.

Here's a summary of how to create a disk with a file system partition, format the file system, copy the `echo` program into the new disk, and then run `echo`, passing argument `iloveos`. (Argument passing won't work until you implement it.)

- It assumes that you've already built the examples in `examples/` and that the current directory is `userprog/build`:

```
$ pintos-mkdisk filesys.dsk --filesys-size=2
$ pintos -- -f -q
$ pintos -p ../../examples/echo -a echo -- -q
$ pintos -- -q run 'echo iloveos'
```

- The three final steps can actually be combined into a single command:

```
$ pintos-mkdisk filesys.dsk --filesys-size=2
$ pintos -p ../../examples/echo -a echo -- -f -q run 'echo
iloveos'
```

- If you don't want to keep the file system disk around for later use or inspection, you can even combine all four steps into a single command. The `--filesys-size=n` option creates a temporary file system partition approximately `n` megabytes in size just for the duration of the `pintos` run. The Pintos automatic test suite makes extensive use of this syntax:

```
$ pintos --filesys-size=2 -p ../../examples/echo -a echo -- -f -q
run 'echo iloveos'
```

- You can delete a file from the Pintos file system using the `rm file` kernel action. e.g. `pintos -q rm file`.
- Also, `ls` lists the files in the file system and `cat file` prints a file's contents to the display.

4.3 How User Programs Work

Pintos can run normal C programs, as long as they fit into memory and use only the system calls you implement.

- Notably, `malloc()` cannot be implemented because none of the system calls required for this project allow for memory allocation.

- Pintos also can't run programs that use floating point operations, since the kernel doesn't save and restore the processor's floating-point unit when switching threads.

The `src/examples` directory contains a few sample user programs.

Please note that since we are not implementing the full set of system calls required by the original Pintos project, many examples in this directory may not be functional. We recommend using the echo example to verify your implementation.

- The Makefile in this directory compiles the provided examples, and you can edit it to compile your own programs as well.

Pintos can load ELF executables with the loader provided for you in `userprog/process.c`.

- ELF is a file format used by Linux, Solaris, and many other operating systems for object files, shared libraries, and executables.
- You can actually use any compiler and linker that output 80x86 ELF executables to produce programs for Pintos. (We've provided compilers and linkers that should do just fine.)

You should realize immediately that, until you copy a test program to the simulated file system,

Pintos will be unable to do useful work.

- You won't be able to do interesting things until you copy a variety of programs to the file system.
- You might want to create a clean reference file system disk and copy that over whenever you trash your filesys.dsk beyond a useful state, which may happen occasionally while debugging.

4.4 Virtual Memory Layout

Virtual memory in Pintos is divided into two regions: user virtual memory and kernel virtual memory.

- User virtual memory ranges from virtual address `0` up to `PHYS_BASE`, which is defined in `threads/vaddr.h` and defaults to `0xc0000000 (3 GB)`.
- Kernel virtual memory occupies the rest of the virtual address space, from `PHYS_BASE` up to 4 GB.

User virtual memory is per-process.

- When the kernel switches from one process to another, it also switches user virtual address spaces by changing the processor's page directory base register (see `pagedir_activate()` in `userprog/pagedir.c`).
- Struct thread contains a pointer to a process's page table.

Kernel virtual memory is global.

- It is always mapped the same way, regardless of what user process or kernel thread is running.
- In Pintos, kernel virtual memory is mapped one-to-one to physical memory, starting at `PHYS_BASE`. That is, virtual address `PHYS_BASE` accesses physical address 0, virtual address `PHYS_BASE + 0x1234` accesses physical address `0x1234`, and so on up to the size of the machine's physical memory.

Useful functions:

- Function: `bool is_user_vaddr (const void *va)`
- Function: `bool is_kernel_vaddr (const void *va)`
    - Returns true if va is a user or kernel virtual address, respectively, false otherwise.
- Function: `void *pagedir_get_page (uint32_t *pd, const void *uaddr)`
    - Looks up the frame mapped to uaddr *in* pd.
    - Returns the kernel virtual address for that frame, if *uaddr* is mapped, or a null pointer if it is not.

A user program can only access its own user virtual memory. An attempt to access kernel virtual memory causes a page fault, handled by `page_fault()` in `userprog/exception.c`, and the process will be terminated. Kernel threads can access both kernel virtual memory and, if a user process is running, the user virtual memory of the running process. However, even in the kernel, an attempt to access memory at an unmapped user virtual address will cause a page fault.

4.4.1 Typical Memory Layout

Conceptually, each process is free to lay out its own user virtual memory however it chooses. In practice, user virtual memory is laid out like this:

```
PHYS_BASE +----------------------------------+
          |            user stack            |
          |                 |                |
          |                 |                |
          |                 V                |
          |          grows downward          |
          |                                  |
          |                                  |
          |                                  |
          |                                  |
          |           grows upward           |
          |                 ^                |
          |                 |                |
          |                 |                |
          +----------------------------------+
          | uninitialized data segment (BSS) |
          +----------------------------------+
          |     initialized data segment     |
          +----------------------------------+
          |           code segment           |
0x08048000 +----------------------------------+
          |                                  |
          |                                  |
          |                                  |
          |                                  |
          |                                  |
        0 +----------------------------------+
```

- In this project, the user stack is fixed in size. Traditionally, the size of the uninitialized data segment can be adjusted with a system call, but you will not have to implement this.
- The code segment in Pintos starts at user virtual address 0x08048000, approximately 128 MB from the bottom of the address space.
- The linker sets the layout of a user program in memory, as directed by a "linker script" that tells it the names and locations of the various program segments. You can learn more about linker scripts by reading the "Scripts" chapter in the linker manual, accessible via info ld.
- To view the layout of a particular executable, run `objdump` (80x86) or `i386-elf-objdump` (SPARC) with the `-p` option.

4.5 80x86 Calling Convention

This section summarizes important points of the convention used for normal function calls

on 32-bit 80x86 implementations of Unix. The calling convention works like this:

- The caller pushes each of the function's arguments on the stack one by one, normally using the `PUSH` assembly language instruction. Arguments are pushed in right-to-left order.
- The stack grows downward: each push decrements the stack pointer, then stores into the location it now points to, like the C expression `*--sp = value`.
- The caller pushes the address of its next instruction (the _return address_) on the stack and jumps to the first instruction of the callee. A single 80x86 instruction, `CALL`, does both.
- The callee executes. When it takes control, the stack pointer points to the return address, the first argument is just above it, the second argument is just above the first argument, and so on.
- If the callee has a return value, it stores it into register `EAX`.

- The callee returns by popping the return address from the stack and jumping to the location it specifies, using the 80x86 `RET` instruction.
- The caller pops the arguments off the stack.

Consider a function `f()` that takes three `int` arguments. This diagram shows a sample stack frame

as seen by the callee at the beginning of step 3 above, supposing that `f()` is invoked as `f(1, 2, 3)`.

The initial stack address is arbitrary:

```
                              +----------------+
                  0xbffffe7c  |        3       |
                  0xbffffe78  |        2       |
                  0xbffffe74  |        1       |
stack pointer --> 0xbffffe70  | return address |
                              +----------------+
```

4.6 Program Startup Details

4.6.1 The Entry Point

The Pintos C library for user programs designates `_start()`, in `lib/user/entry.c`, as the entry point for user programs. This function is a wrapper around `main()` that calls `exit()` if `main()` returns:

```
void
_start (int argc, char *argv[])
{
  exit (main (argc, argv));
}
```

4.6.2 Argument Passing

The kernel must put the arguments for the initial function on the stack before it allows the user program to begin executing.

The arguments are passed in the same way as the normal calling convention (see section 80x86 Calling Convention), and you will implement this in Pintos kernel (see section Argument Passing of Your Tasks).
Consider how to handle arguments for the following example command: `/bin/ls -l foo bar`.

- First, break the command into words: `/bin/ls, -l, foo, bar`.
- Place the words at the top of the stack. Order doesn't matter, because they will be referenced through pointers.

- Then, push the address of each string plus a null pointer sentinel, on the stack, in right-to-left order. These are the elements of `argv`. The null pointer sentinel ensures that `argv[argc]` is a null pointer, as required by the C standard. The order ensures that `argv[0]` is at the lowest virtual address.
- Word-aligned accesses are faster than unaligned accesses, so for best performance, round the stack pointer down to a multiple of 4 before the first push.
- Then, push `argv` (the address of `argv[0]`) and `argc`, in that order.
- Finally, push a fake "return address": although the entry function will never return, its stack frame must have the same structure as any other.

The table below shows the state of the stack right before the beginning of the user program, assuming `PHYS_BASE` is 0xc0000000:

Address	Name	Data	Type
0xbffffffc	`argv[3][...]`	bar\0	`char[4]`
0xbffffff8	`argv[2][...]`	foo\0	`char[4]`
0xbffffff5	`argv[1][...]`	-l\0	`char[3]`
0xbffffffed	`argv[0][...]`	/bin/ls\0	`char[8]`
0xbffffffec	word-align	0	`uint8_t`
0xbffffffe8	`argv[4]`	0	`char *`
0xbffffffe4	`argv[3]`	0xbffffffc	`char *`
0xbffffffe0	`argv[2]`	0xbffffff8	`char *`
0xbffffffdc	`argv[1]`	0xbffffff5	`char *`
0xbffffffd8	`argv[0]`	0xbffffffed	`char *`
0xbffffffd4	`argv`	0xbffffffd8	`char **`
0xbffffffd0	`argc`	4	`int`
0xbffffffcc	return address	0	`void (*) ()`

In this example, the stack pointer would be initialized to 0xbffffffcc.
As shown above, your code should start the stack at the very top of the user virtual address space, in the page just below virtual address `PHYS_BASE` (defined in `threads/vaddr.h`).
You may find the non-standard `hex_dump()` function, declared in `<stdio.h>`, useful for debugging your argument passing code. Here's what it would show in the above example:

```
bffffffc0                                       00 00 00 00 |            ....|
bffffffd0  04 00 00 00 d8 ff ff bf-ed ff ff bf f5 ff ff bf |................|
bffffffe0  f8 ff ff bf fc ff ff bf-00 00 00 00 00 2f 62 69 |............./bi|
bffffff0  6e 2f 6c 73 00 2d 6c 00-66 6f 6f 00 62 61 72 00 |n/ls.-l.foo.bar.|
```

4.7 System Call Details

The first project already dealt with one way that the operating system can regain control from a user program: interrupts from timers and I/O devices.
These are "external" interrupts, because they are caused by entities outside the CPU.
The operating system also deals with software exceptions, which are events that occur in program code.

- These can be errors such as a page fault or division by zero.
- Exceptions are also the means by which a user program can request services ("system calls") from the operating system.

In the 80x86 architecture, the `int` instruction is the most commonly used means for invoking system calls.

- This instruction is handled in the same way as other software exceptions.
- In Pintos, user programs invoke `int $0x30` to make a system call. The system call number and any additional arguments are expected to be pushed on the stack in the normal fashion before invoking the interrupt (see section 80x86 Calling Convention).
- Thus, when the system call handler `syscall_handler()` gets control, the system call number is in the 32-bit word at the caller's stack pointer, the first argument is in the 32-bit word at the next higher address, and so on. The caller's stack pointer is accessible to `syscall_handler()` as the esp member of the `struct intr_frame` passed to it. (`struct intr_frame` is on the kernel stack.)

The 80x86 convention for function return values is to place them in the `EAX` register. System calls that return a value can do so by modifying the `eax` member of `struct intr_frame`.
You should try to avoid writing large amounts of repetitive code for implementing system calls.
Each system call argument, whether an integer or a pointer, takes up 4 bytes on the stack.
You should be able to take advantage of this to avoid writing much near-identical code for retrieving each system call's arguments from the stack.

5. Tasks and Requirements (Lab 2 — User Programs)

5.1 Task 1 — Process Termination Messages

Goal: When a user process terminates, print a standardized exit message exactly once.

Required behavior:

- When a **user process** exits (normal exit or killed due to error), print **exactly**: `"%s: exit(%d)\n"`
- `%s` must be the **program name only** (e.g., `echo`, not the full command line `echo hello`).
- `%d` must be the process's **final exit status** (what the kernel records for that process).
- **Do not print** this message for kernel threads (threads that never ran user code).
- No extra debug prints to stdout/stderr that would break output matching.

What to think about (no solution here)

- Where to store the process “name” (program-only) so it survives until exit.
- How to ensure the message prints **once** even on abnormal termination.
- How to propagate exit status to the parent for `wait()` later (even if you won’t fully implement `wait` now).

Step-by-step guidance (implementation plan)

Step 1: Record the correct thread name

- The `struct thread` already contains a `name` member; however, its existing implementation fails to meet the project specifications.
- Parse the command line to extract the first token (program name).
- To get the correct name, you may use the `strtok_r` function defined in `src/lib/string.c`. Please refer to the comments in the source file for detailed usage instructions to ensure correct implementation.
- Store the name in struct thread.

Step 2: Set and store exit status consistently

- Add an `exit_status` member to the `thread` structure.
- Assign a valid initial value to `exit_status`, and be sure to update it correctly across all future tasks.

Step 3: Print at the single exit point

- Print the message in the central process-exit path (not in many scattered places).
- Ensure kernel threads don’t trigger it (e.g., check `thread_current()->pcb != NULL` or equivalent).

Acceptance checklist

- Correct format and spacing
- Printed **exactly once** per exiting **user** process
- Exit status matches the process termination status
- No message printed for kernel threads

5.2 Task 2 — Argument Passing

Goal: Support passing command-line arguments to user programs via the initial user stack.

Required behavior

- `process_execute(cmd_line)` accepts a full command line string and parses it into:
  - `argv[0]` = program name
  - `argv[1..]` = arguments separated by spaces
  - Multiple spaces behave like a single separator
- When starting the user process, the initial user stack must contain (in standard C ABI layout):
  - argument strings

    - pointers `argv[i]` to those strings
    - `argv[argc] = NULL`
    - `argc`
    - a fake return address
    - proper stack alignment as required by your spec

What to think about (no solution here)

- Here, `cmd_line` is provided by kernel code rather than directly by user syscalls, but you should still make a private kernel copy before parsing or passing it to another thread.
- Where to tokenize safely (copy command line first to avoid races).
- Stack growth direction and layout order (strings then pointers, or vice versa).
- Alignment: use 4-byte stack alignment in this lab.
- **Note that the Pintos file system does not support internal synchronization. Any filesystem operations (like open/read/write) must be protected by appropriate synchronization (e.g., a global lock).**

Step-by-step guidance

Step 1: Copy command line

- Make a private kernel copy of `cmd_line` for parsing and later load, because the original pointer may become invalid or be modified after `thread_create()` due to concurrent execution.

Step 2: Tokenize

- To tokenize the command-line string, you may also use the `strtok_r` function as in Task 1. Please refer to the comments in the source file for detailed usage instructions to ensure correct implementation.

Step 3: Build the user stack

- Push argument strings onto the stack (including `\0`)
- Record each string's user address
- Push `argv[argc] = NULL`
- Push `argv[i]` pointers in reverse order
- Push `argv` (address of argv[0])
- Push `argc`
- Push fake return address
- Apply padding for alignment before pushing pointers

Acceptance checklist

- `argv[0]` is program name
- `argc` is correct
- `argv` pointers and strings are valid user addresses
- `argv[argc] == NULL`
- Stack alignment matches the project spec

5.3 Task 3 — Accessing User Memory Safely

Goal: Prevent kernel crashes when user programs pass invalid pointers to syscalls.

Required behavior

User programs may pass:

- `NULL` pointers
- pointers to unmapped pages
- pointers into kernel virtual address space

Your kernel must:

- **never crash, assert, or panic** due to invalid user pointers
- terminate the offending user process cleanly on invalid memory access
- ensure that any user process terminated by the kernel due to an exception or an invalid user pointer exits with status `-1`.

What to think about (no solution here)

- How to validate a pointer is a user virtual address (`< PHYS_BASE`) and mapped.
- How to safely read syscall args from the user stack without trusting the pointer.
- Difference between validating:
  - a single pointer (e.g., `char *`)
  - a buffer range (`void *buffer, size`)

Step-by-step guidance

Step 1: Create safe "get user byte/word" helpers

- A primitive that attempts to read a byte from user address and returns failure instead of faulting.
- A similar helper can be used when the kernel needs to copy data to user memory safely.

Step 2: Validate syscall arguments before use

- Validate the user stack pointer itself and each argument address passed to syscall.
- For strings: ensure the entire string is readable until `'\0'`.
- For buffers: ensure the entire range `[buffer, buffer+size)` is accessible.

Step 3: Maintain exit status in task 1

- On any invalid user access: terminate process with a defined status (often `-1`).
- Ensure cleanup runs (don't just "return" and leave state inconsistent).

Acceptance checklist

- Invalid pointers never panic kernel

5.4 Task 4.1 — System Call Entry & Dispatch

Goal: Implement syscall handler framework: fetch syscall number/args, validate them, dispatch, and return values.

Required behavior

- Implement a syscall entry point (interrupt handler) that:
  - reads syscall number from the user stack
  - reads syscall arguments from the user stack
  - validates pointers/arguments safely (Task 3)
  - dispatches to the correct syscall implementation
- Never trust user stack pointers: must validate before dereference.

What to think about (no solution here)

- where the syscall number is located on the stack.
- How many args each syscall takes.

Step-by-step guidance

Step 1: Implement "fetch argument" helpers

- A function that safely reads a 32-bit int/pointer value from the user stack (via a user-space address pointer).

Step 2: Implement a dispatch switch

- `switch(syscall_no)` → call corresponding function.
- Default: terminate process for unknown syscalls.

Acceptance checklist

- Correct syscall number extraction
- Correct argument extraction
- Robust pointer/argument validation
- No kernel crash on malformed syscalls

5.5 Task 4.2 — Implement Required System Calls (Subset: process control)

You only need to implement **halt()** and **exit(status)**.

5.5.1 Syscall — halt()

Goal: Shut down the machine.

Required behavior

- **Terminates Pintos by calling** `shutdown_power_off()` (declared in `devices/shutdown.h`).
- No return value.

Acceptance checklist

- Calling `halt()` stops the machine immediately
- No extra prints

5.5.2 Syscall — exit(int status)

Goal: Terminate the current user process with the given status.

Required behavior

- `exit(status)` terminates the calling process.
- Stores `status` as process exit status.
- Triggers Task 1 message printing:
  `"%s: exit(%d)\n"`
- Must not kill the kernel; only the user process should exit.

What to think about (no solution here)

- Make sure exit works even if syscall args are invalid (then exit with -1 per policy).
- Parent/child synchronization for later `wait()` (even if not required now, design shouldn't block it).

Acceptance checklist

- Exit status recorded correctly
- Exit message printed exactly once (Task 1)
- Process resources cleaned
- No deadlocks / kernel crash

5.5.3 Syscall — write

The test cases require the `write` system call to function. We are providing the necessary code for `write` here, although it is **NOT** a requirement for Project 2. To run the tests, please integrate this code into your `syscall_handler` and make slight adjustments to ensure compatibility with your existing implementation.

```
    case SYS_WRITE:
      {
        int fd = (int) copy_in_u32 (f->esp + 4);
        const void *buffer = (const void *) copy_in_u32 (f->esp + 8);
        unsigned size = (unsigned) copy_in_u32 (f->esp + 12);
```

```
          validate_user_range (buffer, size);       /* buffer must be
readable */
          f->eax = syscall_write (fd, buffer, size);
          break;
        }
```

The `copy_in_u32` function was previously implemented in Task 4.1 (Step 1), while `validate_user_range` was completed in Task 3. These function names are not fixed and can be modified to suit your implementation.
The code for `syscall_write` is provided below:

```
/* Writes SIZE bytes from BUFFER to the open file FD.
   Returns the number of bytes actually written, which may be
   less than SIZE if some bytes could not be written. */
int
syscall_write (int fd, const void *buffer, unsigned size)
{
  /* For simplicity, we only handle writing to stdout (fd = 1). */
  if (fd != 1)
    return -1;

  /* Write to console output. */
  putbuf (buffer, size);
  return size;
}
```

6. Recommended Implementation Order (Milestones)

1. Argument Passing (Task 2)
   Many user programs will crash immediately if stack setup is wrong.
2. Minimal syscalls: `halt`, `exit`, and `write`
   Enables basic user-space printing and sanity checks.
3. User memory safety (Task 3)
   Stabilize the kernel against invalid pointers before implementing full I/O.
4. Finish the required syscall subset (Task 4.2)

7. Build & Run: Command Cheat Sheet

7.1 setup

Run the container as you did for Project 1.

7.2 Build and Run Pintos

```
cd /home/PKUOS/pintos/src/userprog
make clean
make
cd build
make check
```

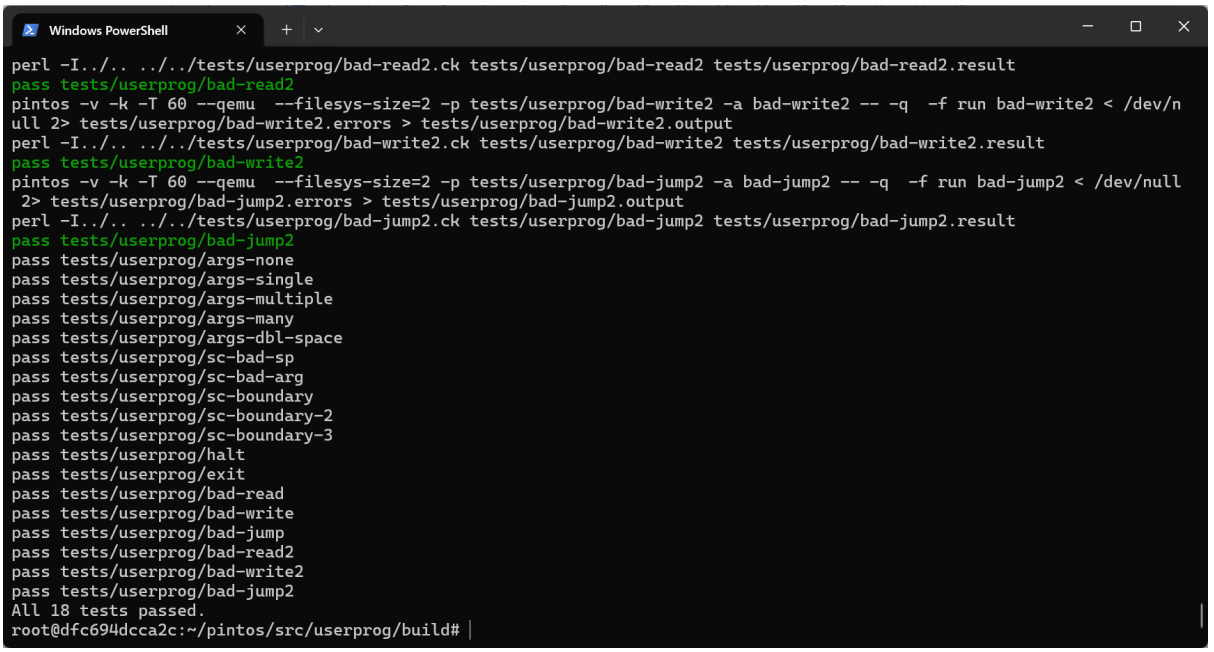
7.3 Code modification

Although `make check` runs successfully on the base code, only a few test cases pass, because the required functionality is not yet implemented. Therefore, you need to modify the source code under `src/userprog/`.

7.4 Rebuild

After the modification is completed, we need to go back to the src/userprog directory, remove the `build` directory and regenerate it.

```
cd /home/PKUOS/pintos/src/userprog
make clean  (It will display "rm -rf build")
make
cd build
make check
```



Note:

```
When rebuilding, you must execute make clean first; otherwise,
test outputs may accumulate, leading to an incorrect number of
reported test cases.
```

8. Common Pitfalls (Short List)

- Immediate page faults on program start: usually argument passing / stack layout is wrong.
- Kernel panics during syscalls: user pointer validation is incomplete.
- Output interleaving across processes: consider writing console output in larger chunks to reduce mixing.
- Deadlocks: file system synchronization must be carefully scoped (avoid holding a global lock while waiting on another event).
- Leaking resources: ensure process resources are cleaned up correctly on exit.

9. Grading

Participation: 4% for 4 lab sessions of Project 2.

Final codes: 6% (10% penalty for each day if submitted late). All test cases have equal weight. You earn credit for a test case if your implementation passes it.

Final report: 5% (10% penalty for each day if submitted late)

Submission: After completing the project, students are required to compress the whole Pintos code into a ZIP file and upload it to Canvas.

Regarding the report, please provide line-by-line comments for all code you modify or add, clearly explain your design decisions, and include a detailed description of your implementation and solution in the final report. The code and the final report should be consistent with one another.